Algorithm AS 222

Resistant Smoothing Using the Fast Fourier Transform

By W. Härdle†

Institut für Wirtschaftstheorie II, West Germany

[Received March 1985. Final revision July 1986]

Keywords: Kernel regression estimation; resistant smoothing; Fast Fourier Transform;

Language

Fortran 66

Description and Purpose

Suppose $\{(X_j, Y_j)\}_{j=1}^n$ are two-dimensional data points and it is desired to compute the regression curve $r(x) = E(Y|X = x)$ of Y on X. The following curve estimator with kernel function K and bandwidth h

$$r_n^*(x) = \frac{d_n(x)}{f_n(x)} = \frac{n^{-1}h^{-1}\sum_{j=1}^{n} K(h^{-1}(x - X_j))Y_j}{n^{-1}h^{-1}\sum_{j=1}^{n} K(h^{-1}(x - X_j))} \tag{1}$$

has been introduced by Nadaraya (1964) and Watson (1964). Brillinger (1977) pointed out the non-resistance to outliers and proposed an M-type smoother. Resistant regression estimates are desirable in data analysis as was pointed out by Velleman and Hoaglin (1981). An application of a resistant smoother to a chemical problem is described in Bussian and Härdle (1984).

In this paper we present an algorithm for the one-step M-type smoother

$$r_n(x) = r_n^*(x) + \frac{n^{-1}h^{-1}\sum_{j=1}^{n} K(h^{-1}(x - X_j))\psi(res_j)}{n^{-1}h^{-1}\sum_{j=1}^{n} K(h^{-1}(x - X_j))\psi'(\mathrm{res}_j)} \tag{2}$$

where $res_j = Y_j - r_n^*(X_j)$ and $\psi(u) = \max\{-c, \min\{u, c\}\}$, $c > 0$ is Huber's (1981) well known psi function. The boundedness of ψ makes r_n resistant against outliers since large residuals are downweighted. Theoretical aspects of resistant nonparametric regression estimators have been considered by Utreras (1981), Cox (1983) and Härdle (1984) among others. The choice of the bandwidth h is important for the kernel method but in this algorithm the choice of h is left to the user. Bandwidth selection procedures for r_n^* are investigated in Härdle and Marron (1985), Hall (1984) for instance. These procedures require an enormous amount of computation, since the regression estimate has to be calculated for a wide range of bandwidths h. It is therefore desirable to have an efficient numerical method for the computation of formula (2). This is achieved here by the use of the fast Fourier transform. The choice of the cutoff parameter c,

† *Address for correspondence*: Institut für Wirtschaftstheorie II, Universität Bonn, Adenauerallee 24–26, D–5300 Bonn, West Germany.

© 1987 Royal Statistical Society 0035–9254/87/36104 $2.00

Downloaded from academic.oup.com/jrssc/article/36/1/104/6985332 by guest on 13 October 2026

that regulates the amount of downweighting of the residuals, is not part of this algorithm, and has to be supplied by the user. To our knowledge, tecniques for adapting the cutoff parameter c are not known.

Numerical Method

The use of the fast Fourier transform in the setting of kernel estimators was suggested by Silverman (1982). Denote the Fourier transformation by F and apply it to d_n, to get

$$F(d_n)(w) = F(K)(hw)D_n(w) \tag{3}$$

where

$$D_n(w) = n^{-1} \sum_{j=1}^{n} \exp(iwX_j)Y_j.$$

For computational efficiency the Gaussian kernel is implemented here, so

$$F(K)(w) = \exp(-\tfrac{1}{2}w^2).$$

We use the Fortran subroutine *FASTF* by Monro (1975), Algorithm AS83, for the efficient evaluation of the complex discrete Fourier transform. In a first step $D_n(w)$ is calculated after discretization of X, then formula (3) is computed and d_n is found by inverse transformation. The density estimate f_n is calculated in a similar way. After the first call the transform $D_n(w)$ is retained, so that subsequent calls with different bandwidths can use D_n. Next, the transformed residuals $\psi(\text{res}_j)$, $\psi'(\text{res}_j)$ are found and the nonlinear one-step correction of $r_n(x)$ is computed in the same way as the numerator of r_n^*.

Underflow is avoided by setting $F(d_n)$ equal to zero if the argument is larger than a constant BIG. The calculations should be done on an interval that is somewhat larger than the range of X in order to avoid "boundary effects" of the *FFT*. The subroutine *RESSMO* can be called in three modes (*NEWCAL* = 0, 1, 2). In the first mode the transform $D_n(w)$ and the estimated curve $r_n(x)$ are computed, in the second mode the estimate is found with the use of the retained $D_n(w)$, in the third mode it is assumed that the same h as in the prevous call is taken but only the cutoff parameter c is changed, so only the one-step correction is to be calculated in this mode.

Structure

SUBROUTINE RESSMO (X,Y,NIN,CUTOFF,XLO,XHI,BANDW,SMOOTH,NOUT, NEWCAL,IWK,W,WA,WB,WC,WK,IFAULT)

Formal parameters

X	Real array (*NIN*)	input: contains the x-data
Y	Real array (*NIN*)	input: contains the y-data
NIN	Integer	input: the sample size
CUTOFF	Real	input: cutoff point of Huber's psi-function, must be greater zero, otherwise *IFAULT* = 1
XLO	Real	input: left boundary point of the interval on the estimate is calculated
XHI	Real	input: right bondary point of the interval on which the estimate is calculated
BANDW	Real	input: the bandwidth
SMOOTH	Real array (*NOUT*)	output: the values of the regression estimate. *SMOOTH*(*I*) is an estimate at *XLO* + (*I* − .5)(*XHI* − *XLO*) *NOUT*

Downloaded from academic.oup.com/jrssc/article/36/1/104/6985332 by guest on 13 October 2026

NOUT	Integer	input: number of points at which the estimate is calculated. *NOUT* must be less or equal to 1024 (in this version) otherwise *IFAULT* = 2
NEWCAL	Integer	input: mode parameter 0: compute *SMOOTH* from *X* and *Y* 1: use previously computed Fourier transforms of the *x*-Data and D_n stored in $WK(3,\cdot) - WK(6,\cdot)$ 2: use previously computed r_n^* stored in $WK(1,\cdot)$
IWK	Integer array (*NIN*)	input: work area
W	Real array (*NIN*)	input: work area
WA,WB,WC	Real array (*NOUT*)	input: work area
WK	Real field (6,*NOUT*)	
	$WK(1,\cdot)$	input: if *NEWCAL* = 2, the values of r_n^* at the same grid points as *SMOOTH* output: the values of r_n^*
	$WK(2,\cdot)$	input: work area
	$WK(3,\cdot)$, $WK(4,\cdot)$	input: if *NEWCAL* ≠ 0, the Fourier transform D_n (real and imaginary part) as previously output. output: the Fourier transform D_n.
	$WK(5,\cdot)$, $WK(6,\cdot)$,	input: if *NEWCAL* ≠ 0, the Fourier transform of the *x*-Data as previously output output: the Fourier transform of the *x*-Data
IFAULT	Integer	output: performance indicator 1: *CUTOFF* is less or equal zero 2: *NOUT* is not a power of two or greater than 1024 3: *XHI-XLO* is less or equal zero 4: *BANDW* is less or equal zero 5: *NEWCAL* is not 0, 1 or 2 <0: indicates how many times $\lvert \text{res}_j \rvert \geqslant c$

Auxiliary routines

Subroutine *RESSMO* calls a subroutine *LINSMO* four times that calculates d_n, f_n as well as the numerator and the denominator of the one-step correction term. Subroutine *FASTF* which performs the forward and reverse discrete Fast Fourier transform is required. This can be the routine of Monro (1975) or equivalent routines.

Restrictions and Remarks

The number *NOUT* must be chosen to be equal to 2^k with k an integer, $3 \leqslant k \leqslant 20$ if the routine *FASTF* of Monro (1975) is used. The present version is limited to *NOUT* equal to 2^k, $3 \leqslant k \leqslant 10$, since there is not much gain (see **Accuracy**) in using very large values of *NOUT* for moderate sample size. The interval at which the estimate is calculated must not be altered between successive calls with *NEWCAL* > 0. Setting *CUTOFF* = ∞ does not give r_n^* since in this algorithm for structural reasons the residuals res_j are centered at $r_n^*(X_j)$ and not at $r_n^*(x)$. The Nadaraya-Watson estimate r_n^* is provided by the array $WK(1,\cdot)$ anyway. A double precision version can be obtained by using double precision in the declaration statements and by replacing the functions *FLOAT*, *SQRT*, etc. by their double precision counterparts.

Downloaded from academic.oup.com/jrssc/article/36/1/104/6985332 by guest on 13 October 2026

Time

After the first call to *RESSMO*, subsequent calls with $NEWCAL > 0$ are performed much faster since $D_n(w)$ need not be computed again. In the even simpler case where with $NEWCAL = 2$ only the one-step correction term is changed by a different *CUTOFF* parameter, r_n^* is left unchanged. If *NEWCAL* equals zero *FASTF* is called eight times: four times to transform d_n, f_n and the correction term; another four times to calculate the inverse transformation. If *NEWCAL* equals one *FASTF* is called six times, since $D_n(w)$ and the transformation of the X-data is retained in $WK(3, \cdot) - WK(6,)$. If *NEWCAL* equals two, *FASTF* is called four times since only the correction term has to be calculated. Table 1 gives timings for the different calling modes ($NEWCAL = 0, 1, 2$) of *RESSMO*. For comparison, the time consumption using formula (2) directly is also presented there. The computations were done on an IBM 3081D in the university computing centre of Bonn. The timings refer to the following data. *NIN* data points were generated, he X-data uniformly distributed over (0, 3), the Y-data with density

$$(9/10)\phi(y - r(x)) + (1/90)\phi((y - r(x))/9),$$

$$r(x) = \sin(\pi x),$$

ϕ denoting the standard normal density.

The *CUTOFF* was set to 1.5, $XLO = -1.0$, $XHI = 4.0$ and $BANDW = 0.2$.

Accuracy

The accuracy and smoothness of r_n^* are dependent on the bandwidth h. In Table 2 two measures of accuracy for r_n^* and r_n are shown as functions of h. The averaged square error (*ASE*) and the maximum absolute deviation (*MAD*) were computed at grid points in the

TABLE 1

Timings in econds for calls to RESSMO with $NEWCAL = 0, 1, 2$ *and for calculations by direct application of the formula* (2)

NOUT	*NIN*	$NEWCAL = 0$	$NEWCAL = 1$	$NEWCAL = 2$	*direct*
64	100	0.0101	0.0055	0.0042	0.1417
64	200	0.0110	0.0056	0.0044	0.2757
128	100	0.0199	0.0103	0.0084	0.2775
128	200	0.0205	0.0104	0.0089	0.5532
512	100	0.0835	0.0403	0.0375	1.0950
512	200	0.0835	0.0405	0.0382	2.1783

TABLE 2

Maximal absolute deviation (MAD) and averaged square error (ASE) of the resistant smoother (RESSMO) and the Nadaraya-Watson estimator (NAD WAT) ($NIN = 200$, $NOUT = 512$, $CUTOFF = 1.5$, $XLO = -1$, $XHI = 4$)

	MAD		*ASE*	
h	*RESSMO*	*NAD WAT*	*RESSMO*	*NAD WAT*
0.15	0.88	1.46	0.135	0.394
0.2	0.772	1.13	0.1	0.292
0.25	0.604	1.15	0.087	0.244
0.3	0.62	1.28	0.089	0.219
0.35	0.769	1.34	0.109	0.208
0.4	0.95	1.34	0.131	0.206
0.45	1.07	1.31	0.155	0.212

Downloaded from academic.oup.com/jrssc/article/36/1/104/6985332 by guest on 13 October 2026

TABLE 3

Discretization error of RESSMO (NIN = 200, CUTOFF = 1.5, XLO = −1, XHI = 4)

h	$NOUT = 512$	$NOUT = 256$
.15	1.9×10^{-2}	9.5×10^{-2}
.2	$.87 \times 10^{-2}$	2.9×10^{-2}
.25	$.51 \times 10^{-2}$	1.8×10^{-2}
.3	$.38 \times 10^{-2}$	1.7×10^{-2}
.35	$.35 \times 10^{-2}$	1.4×10^{-2}
.4	$.34 \times 10^{-2}$	1.1×10^{-2}
.45	$.34 \times 10^{-2}$	$.95 \times 10^{-2}$

interval (0. , 3.) Table 2 shows the advantage of the one-step smoother r_n over the Nadayara-Watson estimator r_n^*, both the *ASE* and the *MAD* are considerably smaller for r_n than for r_n^*. In Table 3 the discretization error of r_n for the same data set that was used above is shown. The errors given are the maximum over the interval (0, 3) of the difference between the estimate and the exact values obtained by direct evaluation of formula (2).

References

Brillinger, D. R. (1977) In Discussion of "Consistent Nonparametric Regression" by C. J. Stone. *Ann. Statist.*, **5**, 622–623.

Bussian, B. and Härdle, W. (1984) Robust smoothing applied to white noise and single outlier contaminated Raman spectra. *Appl. Spectroscopy*, **38**, 309–313.

Cox, D. D. (1983) Asymptotics for *M*-type smoothing splines. *Ann. Statist.*, **11**, 530–551.

Härdle, W. (1984) Robust regression function estimation. *J. Mult. Anal.*, **14**, 169–180.

Härdle, W. and Marron, S. (1985) Optimal bandwidth selection in nonparametric kernel regression. *Ann. Statist.*, **13**, 1465–1481.

Hall, P. (1984) Asymptotic properties of integrated square error and cross-validation for kernel estimation of a regression function. *Z. Wahrscheinlichkeitstheorie*, **67**, 175–196.

Huber, P. J. (1981) *Robust Statistics.* New York: Wiley.

Monro, D. M. (1975) Algorithm AS83. Complex discrete fast Fourier transform. *Appl. Statist.*, **24**, 268–272.

Nadaraya, E. A. (1964) On estimating regression. *Theor. Prob. Appl.*, **9**, 141–142.

Silverman, B. W. (1982) Kernel density estimation using the fast Fourier transform. *Appl. Statist.*, **31**, 93–97.

Utreras, F. (1981) On computing robust splines and applications. *SIAM J. Sci. Stat. Comp.*, **2**, 153–163.

Velleman, P. F. and Hoaglin, D. C. (1981). *Applications, Basics and Computing of Exploratory Data Analysis.* San Francisco: Wadsworth.

Watson, G. S. (1964) Smooth regression analysis. *Sankhya*, A, **26**, 359–372.


```
      SUBROUTINE RESSMO(X, Y, NIN, CUTOFF, XLO, XHI, BANDW, SMOOTH,
     *  NOUT, NEWCAL, IWK, W, WA, WB, WC, WK, IFAULT)
C
C        ALGORITHM AS222  APPL. STATIST. (1987) VOL. 36, NO. 1
C
      INTEGER IWK(NIN)
      REAL X(NIN), Y(NIN), SMOOTH(NOUT), W(NIN), WA(NOUT), WB(NOUT),
     *  WC(NOUT), WK(6, NOUT)
C
      DATA ZERO, ONE, TWO, PI, BIG /0.0, 1.0, 2.0, 3.1415, 30.0/
C
      RANGE = XHI - XLO
C
C        CHECK CUTOFF PARAMETER
C
      IF (CUTOFF .GT. ZERO) GOTO 1
      IFAULT = 1
      RETURN
```

Downloaded from academic.oup.com/jrsssc/article/36/1/104/6985332 by guest on 13 October 2026

```
C         CHECK IF NOUT IS POWER OF TWO
C
     1 NPOW = 8
       M = 2 ** 4
       DO 2 K = 3, 11
       IF (NPOW .EQ. NOUT) GOTO 3
       M = M * 2
     2 NPOW = NPOW + NPOW
       IFAULT = 2
       RETURN
C
C         CHECK RANGE
C
     3 IF (RANGE .GT. ZERO) GOTO 31
       IFAULT = 3
       RETURN
C
C         CHECK BANDWIDTH
C
    31 IF (BANDW .GT. ZERO) GOTO 32
       IFAULT = 4
       RETURN
C
C         CHECK NEWCAL
C
    32 IF (NEWCAL .EQ. 0 .OR. NEWCAL .EQ. 1 .OR. NEWCAL .EQ. 2) GOTO 4
       IFAULT = 5
       RETURN
C
     4 XSTEP = RANGE / FLOAT(NOUT)
       A = FLOAT(NOUT) / (RANGE * FLOAT(NIN))
       N2 = NOUT / 2
       B = TWO * (PI * BANDW / RANGE) ** 2
       IF (NEWCAL .EQ. 2) GOTO 8
C
       JHI = SQRT(BIG / B)
       JMAX = MIN0(N2 - 1, JHI)
       XL01 = XL0 - XSTEP
       IF (NEWCAL .NE. 0) GOTO 51
       DO 5 I = 1, NIN
       IWK(I) = (X(I) - XL01) / XSTEP
     5 W(I) = ONE
    51 CONTINUE
C
C         REFRESH FORMER RESULTS
C
       IF (NEWCAL .NE. 1) GOTO 53
       DO 52 J = 1, NOUT
       WA(J) = WK(3, J)
    52 WB(J) = WK(4, J)
    53 CONTINUE
C
C         FIND NUMERATOR OF NADARAYA-WATSON ESTIMATE
C
       CALL LINSMO(Y, IWK, NIN, SMOOTH, NOUT, NEWCAL, WA, WB, WC, N2,
      *  JHI, JMAX, A, B, M)
C
C         TRANSFER RESULTS TO WORKAREA
C
       DO 54 J = 1, NOUT
       WK(3, J) = WA(J)
       WK(4, J) = WB(J)
    54 WK(1, J) = SMOOTH(J)
C
C         REFRESH FORMER RESULTS
C
       IF (NEWCAL .NE. 1) GOTO 56
       DO 55 J = 1, NOUT
       WA(J) = WK(5, J)
    55 WB(J) = WK(6, J)
    56 CONTINUE
```

Downloaded from academic.oup.com/jrssc/article/36/1/104/6985332 by guest on 13 October 2026

```
C         FIND DENSITY ESTIMATE OF MARGINAL DISTRIBUTION OF X
C
      CALL LINSMO(W, IWK, NIN, SMOOTH, NOUT, NEWCAL, WA, WB, WC, N2,
     *  JHI, JMAX, A, B, M)
C
C         COMPUTE NADARAYA-WATSON ESTIMATE
C
      DO 7 J = 1, NOUT
      TEMP = ZERO
      IF (SMOOTH(J) .GT. ZERO) TEMP = WK(1, J) / SMOOTH(J)
    7 WK(1, J) = TEMP
C
C         COMPUTE HUBER'S PSI FROM RESIDUALS
C
    8 NC = 0
      DO 11 I = 1, NIN
      RES = Y(I) - WK(1, IWK(I))
      IF (RES .GT. (-CUTOFF)) GOTO 9
      NC = NC + 1
      W(I) = -CUTOFF
      GOTO 11
    9 IF (RES .LT. CUTOFF) GOTO 10
      NC = NC + 1
      W(I) = CUTOFF
      GOTO 11
   10 W(I) = RES
   11 CONTINUE
C
C         COMPUTE NONLINEAR CORRECTION
C
      IFAULT = -NC
      ICAL = 0
C
      CALL LINSMO(W, IWK, NIN, SMOOTH, NOUT, ICAL, WA, WB, WC, N2, JHI,
     *  JMAX, A, B, M)
C
C         STORE RESULTS
C
      DO 111 J = 1, NOUT
  111 WK(2, J) = SMOOTH(J)
C
C         DERIVATIVE OF HUBER'S PSI FROM RESIDUALS
C
      DO 112 I = 1, NIN
      TEMP = ZERO
      IF (W(I) .LT. CUTOFF .AND. W(I) .GT. -CUTOFF) TEMP = ONE
  112 W(I) = TEMP
C
C         COMPUTE DENOMINATOR OF NONLINEAR CORRECTION
C
      CALL LINSMO(W, IWK, NIN, SMOOTH, NOUT, ICAL, WA, WB, WC, N2, JHI,
     *  JMAX, A, B, M)
C
C         COMPUTE THE FULL ESTIMATOR
C
      DO 13 J = 1, NOUT
      TEMP = WK(1, J)
      IF (SMOOTH(J) .GT. ZERO) TEMP = TEMP + WK(2, J) / SMOOTH(J)
   13 SMOOTH(J) = TEMP
C
      RETURN
      END

      SUBROUTINE LINSMO(Y, IWK, NIN, SMOOTH, NOUT, NEWCAL, WA, WB, WC,
     *  N2, JHI, JMAX, A, B, M)
C
C         ALGORITHM AS222  APPL. STATIST. (1987) VOL. 36, NO. 1
C
      INTEGER IWK(NIN), ITYPE
      REAL Y(NIN), SMOOTH(NOUT), WA(NOUT), WB(NOUT), WC(NOUT)
C
      DATA ZERO /0.0/
```

Downloaded from academic.oup.com/jrssc/article/36/1/104/6985332 by guest on 13 October 2026

```
      IF (NEWCAL .NE. 0) GOTO 30
C
C        TRANSFORM
C
      DO 10 J = 1, NOUT
      WA(J) = ZERO
   10 WB(J) = ZERO
C
      DO 20 I = 1, NIN
      JI = IWK(I)
      IF (JI .LT. 1 .OR. JI .GT. NOUT) GOTO 20
      WA(JI) = WA(JI) + A * Y(I)
C
   20 CONTINUE
      M1 = M / 2
      ITYPE = 1
C
      CALL FASTF(WA, WB, M1, ITYPE)
C
C        FILTER FOURIER TRANSFORM
C
   30 SMOOTH(1) = WA(1)
      WC(1) = WB(1)
      DO 40 J = 1, JMAX
      C = EXP(-B * FLOAT(J * J))
C
      J1 = J + 1
      J2 = NOUT - J + 1
      SMOOTH(J1) = C * WA(J1)
      WC(J1) = C * WB(J1)
      SMOOTH(J2) = C * WA(J2)
   40 WC(J2) = C * WB(J2)
C
C        UNDERFLOW CORRECTION
C
      IF (JHI + 1 - N2) 60, 70, 50
C
   50 SMOOTH(N2 + 1) = EXP(-B * FLOAT(N2 * N2)) * WA(N2 + 1)
      WC(N2 + 1) = EXP(-B * FLOAT(N2 * N2)) * WB(N2 + 1)
      GOTO 80
C
   60 JHI2 = JHI + 2
      DO 61 J1 = JHI2, N2
      J2 = NOUT - J1 + 2
      SMOOTH(J1) = ZERO
      WC(J1) = ZERO
      SMOOTH(J2) = ZERO
   61 WC(J2) = ZERO
C
   70 SMOOTH(N2 + 1) = ZERO
      WC(N2 + 1) = ZERO
C
   80 CONTINUE
      ITYPE = -1
C
      CALL FASTF(SMOOTH, WC, M1, ITYPE)
C
      RETURN
      END
```

Downloaded from academic.oup.com/jrssc/article/36/1/104/6985332 by guest on 13 October 2026